

MACHINE LEARNING · REGRESSION

Linear Regression: From Concept to Code

Bridging the statistics deck with real scripts, data, and results

What Is Regression?

We want to predict a quantitative response Y from a predictor X

General model: $Y = f(X) + e$

f = the systematic relationship; e = random error, independent of X , mean zero

Linear regression assumes the simplest possible f : a straight line

Simple Linear Regression

$$Y \approx \beta_0 + \beta_1 X$$

β_0 = intercept, β_1 = slope

Once fit, predict with: $\hat{y} = \beta_0 + \beta_1 x$

Everything from here is about estimating β_0 and β_1 from data

Estimating the Best Line

Residual: $e_i = y_i - \hat{y}_i$

Residual Sum of Squares: $RSS = \sum e_i^2$

Squaring removes the sign and penalizes big misses more

Least squares picks β_0, β_1 that minimize RSS

Our Toy Dataset

data_2026.csv: X, Y, Z with a known, perfectly linear relationship

$Y = 2 \cdot X$ exactly, $Z = X/8$ exactly — no noise

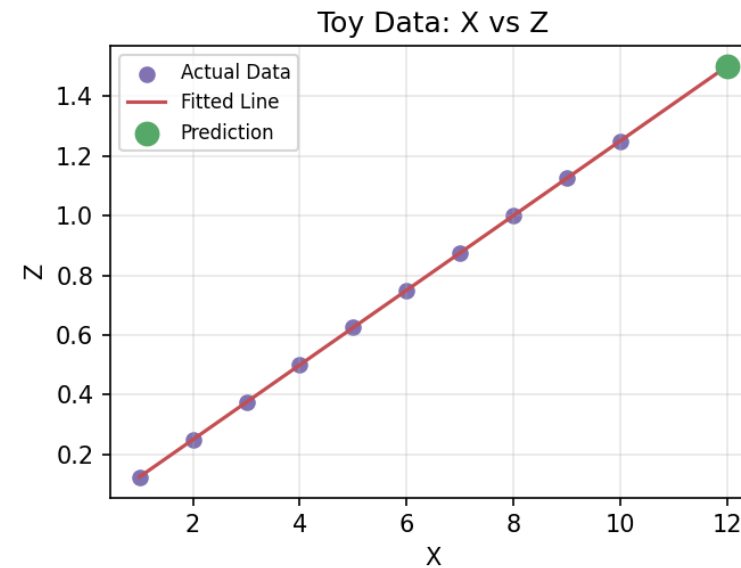
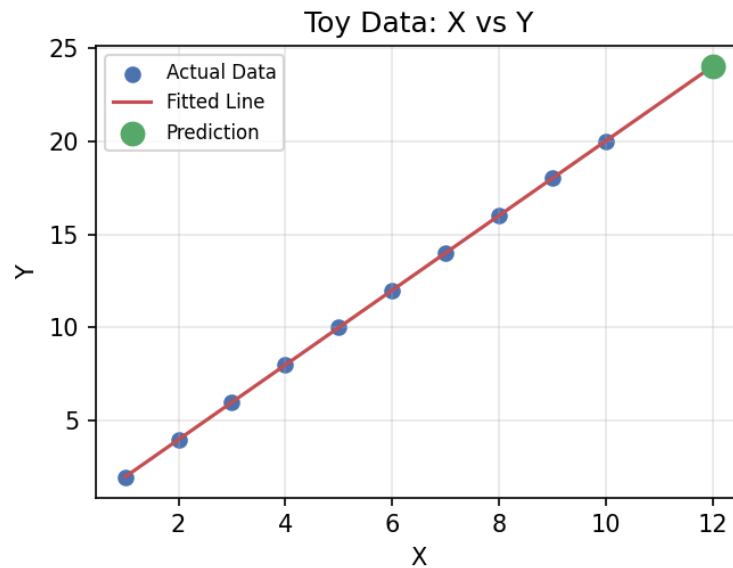
Ideal for checking the code recovers the exact truth

Same idea as the theory deck's own worked example: start with a dataset where the answer is already known

X	Y	Z
1	2	0.125
2	4	0.25
3	6	0.375
4	8	0.5
5	10	0.625

Code: Fit & Predict

```
from sklearn.linear_model import LinearRegression  
model_y = LinearRegression().fit(X, Y)  
model_z = LinearRegression().fit(X, Z)  
Fitted:  $Y = 2.00 \cdot X + 0.00$ ,  $Z = 0.125 \cdot X + 0.00$   
Matches the known truth exactly
```



Getting the Statistics Right

The theory deck goes further: std. error, t-statistic, p-value, confidence intervals for β_0/β_1

Our first pass (v1 notebook) got this wrong — called sklearn's `.score()` expecting those values; it only returns R^2

Fixed in v2 using `statsmodels.OLS` — the same estimator behind the theory deck's math

OLS Summary — Both Models

Model	coef	std err	t	p-value	R ²
$Y = \beta \cdot X$	2.000	3.3e-16	6.0e+15	<0.001	1.000
$Z = \beta \cdot X$	0.125	2.1e-17	6.0e+15	<0.001	1.000

From Toy Data to Real Data: Advertising

Perfect data is a teaching tool — real data has noise

Same dataset used throughout the theory deck: TV / radio / newspaper spend → sales

Start simple, exactly like the theory deck does: does TV spend alone explain sales?

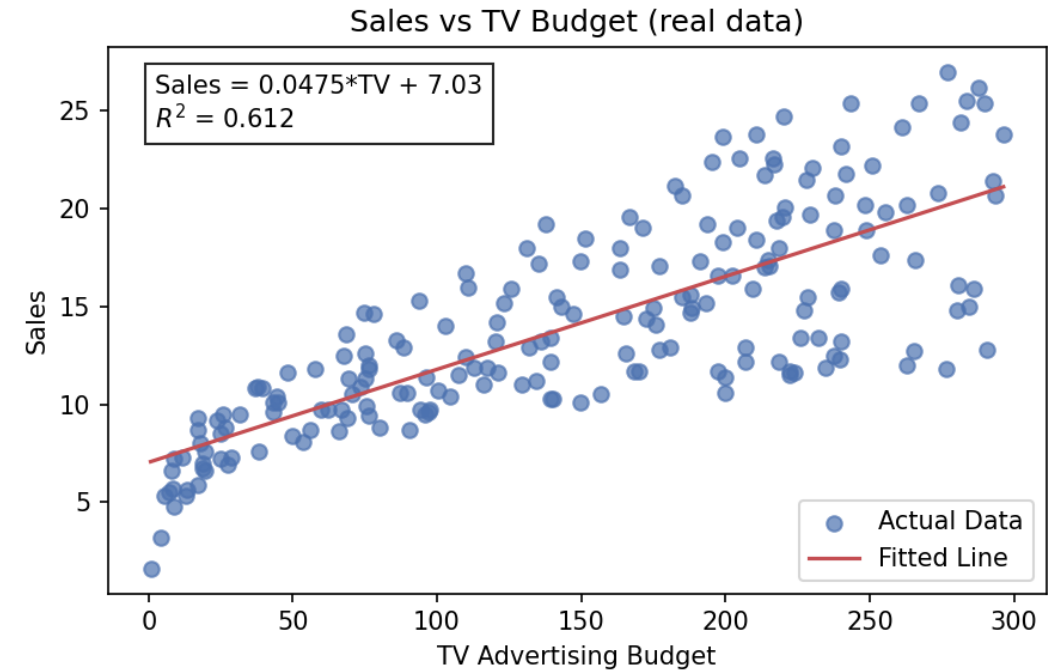
THE CODE

Code: TV → Sales

```
model = LinearRegression().fit(X[['TV']], sales)
```

Fitted: Sales = 0.0475·TV + 7.03

$R^2 = 0.612$ — TV alone explains ~61% of the variance in sales



Interpreting R^2 and Fit Quality

R^2 = fraction of variance in Y explained by the model (0 to 1)

0.612 is decent for one variable, but leaves ~39% unexplained

That gap is exactly what motivates adding radio and newspaper — Multiple Linear Regression, next

Key Takeaways

$Y = f(X) + e$, and linear regression assumes f is a straight line

Least squares finds the β that minimizes RSS — same idea for perfect toy data or noisy real data

Always check the real statistics (std err, t, p, R^2) — not just the fitted line

Full code: `LinearRegressionLearn_v2.ipynb`, `LinearML_training.ipynb`